

DocuTrust: Enterprise Advanced RAG Platform with Automated Self-Correction

Project Description:

System Overview

DocuTrust is an enterprise-grade, self-correcting Retrieval-Augmented Generation (RAG) platform designed to eliminate hallucinations and unsourced answers when parsing complex corporate policy documents. Unlike basic AI document search portals, DocuTrust validates the quality of retrieved information before it is ever shown to the user, and every answer is backed by strict, traceable citations.

At its core, the system implements a **Corrective RAG (CRAG)** design pattern using a multi-agent pipeline. A retrieval agent extracts relevant structural chunks from uploaded documents, a grading agent independently verifies the relevance and quality of those chunks using a local cross-encoder model, and—when retrieval is judged insufficient—a query-rewriting agent triggers a web fallback search to fill the knowledge gap. Only validated, well-sourced content is passed to the generation layer for the final answer.

The backend is built with FastAPI and a text embedding pipeline for semantic search, while MongoDB stores per-client profiles, document metadata, structural indexing lists, and dynamic interaction trace logs for full auditability. The frontend is a sleek, split-pane corporate interface where users drag and drop multi-page PDFs on one side, watch the agent evaluation pipeline execute step-by-step in real time, and receive validated, citation-backed answers on the other.

Problem Domain

Enterprise document search tools built on basic LLM pipelines frequently "hallucinate" — producing confident-sounding answers that are factually wrong, unsourced, or based on irrelevant chunks of text. In regulated and policy-driven environments (HR policies, compliance manuals, legal contracts), this is unacceptable: every answer must be traceable to a specific document and validated before being shown. Existing tools rarely grade retrieval quality, rarely fall back gracefully when local knowledge is insufficient, and rarely expose the reasoning process to the end user.

Core Technology Used

- FastAPI (Python backend) for API handling, embedding pipelines, and orchestration of the CRAG workflow
- LangGraph or CrewAI for building the Corrective RAG (CRAG) multi-agent pipeline
- A local cross-encoder model for grading retrieval relevance before content is displayed
- Vector embedding and semantic search pipeline for document chunk retrieval
- MongoDB for client profiles, document metadata, structural indexing, and interaction trace logs
- Web search fallback integration for query-rewriting when local retrieval is insufficient
- HTML, CSS, and JavaScript for a sleek, split-pane corporate frontend
- Python (3.10+) and Node.js (18+) as core development environments

Key Capabilities

- Self-correcting Corrective RAG (CRAG) pipeline that grades retrieval quality before responding
- Multi-agent architecture: Retriever Agent, Grading Agent, Query-Rewriter Agent, and Generation layer
- Local cross-encoder–based relevance scoring to filter out irrelevant or low-quality chunks
- Automatic web search fallback when document retrieval is insufficient
- Strict citation enforcement — every answer is mapped back to its source document and chunk
- Real-time, step-by-step visualization of agent evaluation logs during query processing
- Multi-page PDF ingestion with structural chunking and metadata extraction
- Per-client document profiles and dynamic interaction trace logging in MongoDB
- Secure, split-pane enterprise UI for document upload and validated Q\&A
- Designed for high-stakes, policy-driven document environments requiring auditability

Target Users

- Enterprises and corporate teams managing internal policy and compliance documentation
- HR, Legal, and Compliance departments requiring traceable, sourced answers
- Knowledge management and IT teams building secure internal document search
- Data scientists and AI engineers exploring Corrective RAG and self-correcting pipelines
- Auditors and reviewers requiring transparent, citation-backed AI responses

Application Scenarios:

Scenario 1: Validated Policy Question Answering

A user drags and drops a multi-page corporate policy PDF into the platform and asks a question such as "What is the maximum carryover for unused vacation days?" The Retriever Agent pulls relevant chunks, the Grading Agent checks them against a relevance threshold using the local cross-encoder model, and only validated chunks are used to generate the answer. The response is returned with strict citations pointing to the exact page and section of the source document.

Scenario 2: Automatic Fallback on Insufficient Local Knowledge

A user asks a question that is not adequately covered by the uploaded documents. The Grading Agent detects that the retrieved chunks fail the relevance threshold, and the Query-Rewriter Agent reformulates the query and triggers a web fallback search. The system clearly indicates in the real-time evaluation log that local retrieval was insufficient and that supplementary information was sourced externally, keeping the user fully informed of the answer's provenance.

Scenario 3: Real-Time Agent Transparency for Compliance Review

A compliance officer uploads a sensitive policy packet and wants to verify how the system arrives at its answers before trusting it in production. As queries are run, the split-pane UI streams step-by-step logs of each agent's evaluation — chunk retrieval, relevance grading, and (if triggered) query rewriting — giving the reviewer full visibility into the self-correction process before the validated answer is displayed.

Technical Architecture:

Component Interaction & Data Flow:

- **Frontend (HTML/CSS/JS Split-Pane UI):** Provides drag-and-drop PDF upload on one pane and displays real-time agent evaluation logs alongside validated, cited answers on the other
- **API Layer (FastAPI):** Receives document uploads and queries, validates input, and orchestrates the CRAG pipeline execution
- **Pipeline Manager:** Coordinates the sequence of Retriever, Grading, and Query-Rewriter agents, manages state, and streams evaluation events to the frontend
- **Retriever Agent:** Extracts relevant structural chunks from vector-embedded document data based on the user's query
- **Grading Agent:** Validates retrieval quality using a local cross-encoder model before any content is allowed downstream
- **Query-Rewriter Agent:** Reformulates the query and triggers a web fallback search when local retrieval is graded as insufficient
- **Embedding & Vector Search Service:** Generates and queries text embeddings for semantic chunk retrieval from uploaded documents
- **Citation Mapping Service:** Maps validated chunks back to exact document sources to enforce strict, traceable citations
- **Database (MongoDB):** Stores client profiles, document metadata, structural indexing lists, and dynamic interaction trace logs
- **External Web Fallback Source:** Supplies supplementary information when document-based retrieval fails the relevance grade
- **Output Layer:** Returns the validated, cited answer along with agent logs and confidence indicators to the frontend

Pre-requisites:

Software Requirements

- Python 3.10+ (virtual environment recommended)
- Node.js 18+ with npm or yarn (if using a JS-based build step for the frontend)
- MongoDB (local instance or hosted cluster)
- Git for version control
- Visual Studio Code or any modern IDE
- Web browser (Chrome/Firefox) for frontend access

Libraries / Frameworks / Dependencies

- FastAPI, Uvicorn (backend server)
- LangGraph or CrewAI for the CRAG agent pipeline
- A local cross-encoder model library (e.g., sentence-transformers) for relevance grading
- PyMongo / Motor for MongoDB integration

- Requests / HTTPX for web fallback API communication
- PDF parsing library for multi-page document ingestion and chunking
- HTML, CSS, and vanilla JavaScript (or a lightweight framework) for the split-pane UI
- Install using: `pip install -r requirements.txt`

Hardware Requirements

- Minimum: 8 GB RAM, CPU-based execution supported
- Recommended: 16 GB RAM (or GPU) for faster cross-encoder grading on large documents
- Stable internet connection for web fallback retrieval and external API access

Prior Knowledge Required:

- Basic understanding of Python programming and virtual environments
- Familiarity with web development concepts (HTML, CSS, JavaScript)
- Knowledge of REST APIs and JSON-based communication
- Fundamental understanding of RAG (Retrieval-Augmented Generation) and vector embeddings
- Awareness of backend frameworks like FastAPI and document databases like MongoDB
- Basic command-line usage (Linux/Windows) and Git version control

Project Objectives:

- Build a self-correcting Corrective RAG (CRAG) pipeline that grades retrieval quality before generating answers
- Implement a multi-agent architecture with Retriever, Grading, and Query-Rewriter agents
- Use a local cross-encoder model to validate document relevance prior to response generation
- Enforce strict, traceable citations for every generated answer
- Provide a web fallback mechanism for queries not adequately covered by uploaded documents
- Store client profiles, document metadata, and interaction logs in MongoDB for auditability
- Design a sleek, split-pane enterprise UI with real-time agent evaluation visualization
- Ensure the system is robust against hallucination and unsourced text generation
- Optimize the pipeline for secure, scalable enterprise document workflows

System Workflow:

1. Document Upload:

Users drag and drop multi-page corporate policy PDFs onto the left pane of the interface. The system extracts text, performs structural chunking, and generates embeddings for semantic search.

2. Request Submission:

The user submits a natural-language question through the UI. The frontend sends the query, along with relevant client/session context, to the FastAPI backend.

3. Pipeline Initialization:

The backend validates the request and the Pipeline Manager initializes the CRAG workflow, preparing the Retriever, Grading, and Query-Rewriter agents.

4. Chunk Retrieval (Retriever Agent):

The Retriever Agent queries the vector embedding index to fetch the most semantically relevant structural chunks from the uploaded documents.

5. Relevance Grading (Grading Agent):

The Grading Agent uses a local cross-encoder model to score each retrieved chunk for relevance and quality, filtering out anything below the validation threshold.

6. Conditional Fallback (Query-Rewriter Agent):

If the graded chunks are insufficient, the Query-Rewriter Agent reformulates the query and triggers a web search fallback to supplement the missing information.

7. Citation Mapping:

Validated chunks (and any fallback content) are mapped to their exact source documents and assigned structured citations.

8. Answer Generation:

The system generates the final answer strictly from validated, cited content, avoiding any unsupported or hallucinated text.

9. Response Assembly:

The backend aggregates the final answer, citations, agent evaluation logs, and confidence metrics into a structured response.

10. Visualization and Output:

The frontend displays the validated answer with citations on the right pane, while the left/center pane shows the step-by-step agent evaluation log for full transparency.

Milestone 1: Environment Setup and Project Initialization

Activity 1.1: Project Setup and Dependency Installation

This activity establishes the foundational environment required to run the DocuTrust platform. A clean, isolated setup ensures that backend, database, and frontend components work without conflicts.

- Create a Python virtual environment for backend isolation
- Install backend dependencies using requirements.txt
- Set up a local or hosted MongoDB instance
- Initialize the frontend split-pane HTML/CSS/JS structure
- Verify project folder structure (backend, frontend, services, etc.)

Activity 1.2: Environment Variable Configuration

This activity ensures secure handling of sensitive credentials such as LLM API keys and MongoDB connection strings.

- Create a .env file in the project root
- Add LLM/embedding API keys required for generation and embeddings
- Add MONGODB_URI for database connectivity
- Use environment loaders (e.g., dotenv) in the backend
- Ensure .env is excluded via .gitignore

Activity 1.3: Backend Server Initialization and Verification

This activity validates that the FastAPI backend is correctly configured and operational.

- Run backend using `uvicorn main:app --reload`
- Verify server is running on `http://localhost:8000`
- Test `/api/health` endpoint for status check
- Confirm API documentation via `/docs` (Swagger UI)
- Ensure no startup errors in logs

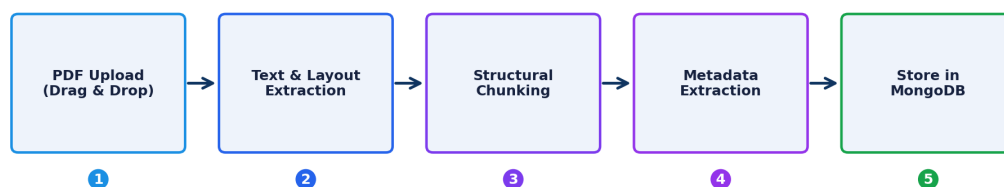
Activity 1.4: Frontend Initialization and Connectivity Check

This activity ensures the split-pane frontend is properly connected to backend services.

- Serve the frontend (static files or a lightweight dev server)
- Configure API base URL for backend communication
- Test basic upload and query request-response flow from UI
- Verify UI loads without errors

Milestone 2: Document Ingestion and Vector Pipeline

Document Ingestion Pipeline



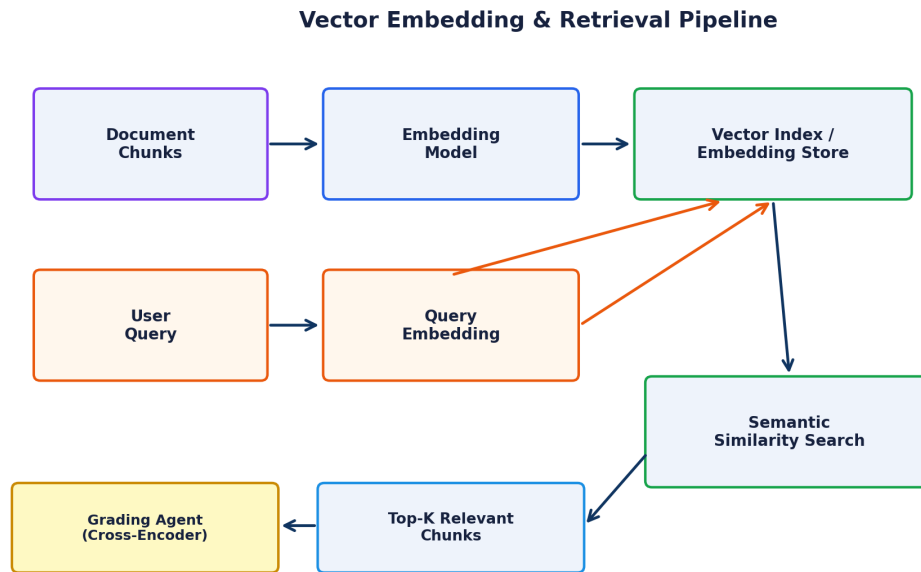
Activity 2.1: PDF Upload and Structural Chunking

This activity implements multi-page PDF ingestion, extracting text and splitting it into structurally meaningful chunks for retrieval.

- Implement drag-and-drop PDF upload on the frontend
- Extract text and structural metadata from multi-page PDFs

- Split documents into chunks suitable for embedding and retrieval
- Store document metadata and structural indexing lists in MongoDB

Activity 2.2: Embedding Pipeline Integration



This activity builds the text embedding pipeline used for semantic chunk retrieval.

- Integrate a text embedding model into the FastAPI backend
- Generate and store embeddings for each document chunk
- Build a vector search index for semantic similarity queries
- Validate retrieval accuracy with sample queries

Activity 2.3: Client Profile and Trace Logging

This activity sets up per-client data isolation and auditability.

- Design MongoDB schema for client profiles
- Implement dynamic interaction trace logging for each query
- Ensure document metadata is linked to the correct client profile
- Provide a basic mechanism to review historical interaction logs

Milestone 3: Corrective RAG (CRAG) Agent Pipeline

Activity 3.1: Multi-Agent Architecture Design

This activity designs the core CRAG multi-agent architecture using LangGraph or CrewAI.

- Define agent roles: Retriever, Grading, Query-Rewriter
- Establish pipeline flow and conditional branching logic
- Ensure modular design for independent agent execution
- Design system to support future agent additions

Activity 3.2: Retriever Agent Implementation

This activity implements the agent responsible for fetching relevant structural chunks.

- Implement vector similarity search for chunk retrieval
- Rank retrieved chunks by relevance score
- Pass structured chunk data to the Grading Agent
- Handle cases with zero or low-confidence retrieval results

Activity 3.3: Grading Agent and Cross-Encoder Integration

This activity implements the relevance validation step that prevents hallucinated or unsourced answers.

- Integrate a local cross-encoder model for relevance scoring
- Define and tune a relevance threshold for chunk validation
- Filter out chunks that fail the relevance check
- Log grading decisions for transparency and debugging

Activity 3.4: Query-Rewriter Agent and Web Fallback

This activity implements the self-correction mechanism that activates when local retrieval is insufficient.

- Detect insufficient retrieval based on Grading Agent output
- Reformulate the original query for improved web search results
- Integrate a web search fallback source
- Merge fallback content with validated local content where applicable

Activity 3.5: Strict Citation Enforcement

This activity ensures every generated answer is traceable to its source.

- Map each validated chunk to its source document and location
- Enforce that generated answers only use cited, validated content
- Format citations for clear display in the UI
- Reject or flag any generation output lacking proper citation support

Milestone 4: Frontend Development and Backend Integration

Activity 4.1: Split-Pane UI Design

This activity builds the sleek, corporate split-pane interface central to the DocuTrust experience.

- Design left pane for multi-page PDF drag-and-drop upload
- Design right pane for validated, cited answer display
- Ensure responsive, professional corporate styling
- Maintain clean navigation and user flow

Activity 4.2: Real-Time Agent Evaluation Log Streaming

This activity enables users to watch the CRAG pipeline execute step by step.

- Implement streaming of agent evaluation events from backend to frontend
- Display retrieval, grading, and fallback events in real time
- Visually distinguish validated chunks from rejected ones
- Improve transparency and trust through live pipeline visibility

Activity 4.3: Frontend-to-Backend API Integration

This activity establishes reliable communication between the UI and backend services.

- Configure API base URL for the frontend
- Send document upload and query requests to FastAPI endpoints
- Handle request payloads (documents, queries, client context)
- Manage API responses and error handling

Activity 4.4: Citation and Answer Rendering

This activity ensures the final validated answer is presented clearly and professionally.

- Render the generated answer with inline citation markers
- Display source document and chunk references for each citation
- Show confidence indicators based on grading results
- Improve overall UI/UX for enterprise usability

MILESTONE 5: TESTING & VALIDATION

Activity 5.1: Test Case Design and Execution

This activity validates core functionality using well-defined test cases, including valid queries, insufficient-retrieval scenarios, and error conditions.

- Validate full CRAG pipeline execution with all agents
- Test fallback behavior when local retrieval is insufficient
- Check handling of invalid or empty document uploads
- Verify streaming evaluation log behavior
- Ensure proper error messages for failures

Activity 5.2: Unit Testing of Core Components

Unit testing verifies the correctness of individual modules independently.

- Test Retriever Agent for correct chunk retrieval
- Validate Grading Agent scoring against the cross-encoder model
- Test Query-Rewriter Agent for proper fallback triggering
- Check citation mapping accuracy
- Ensure proper handling of edge cases

Activity 5.3: Integration Testing

This activity verifies that all system components work together seamlessly.

- Test full workflow (UI → Backend → Agents → Output)
- Verify API request and response handling
- Ensure streaming data reaches frontend correctly
- Validate coordination between Retriever, Grading, and Query-Rewriter agents
- Confirm no runtime errors during execution

Activity 5.4: User Acceptance Testing (UAT)

User Acceptance Testing evaluates the system from an enterprise end-user perspective.

- Evaluate ease of use of the split-pane UI
- Verify clarity and trustworthiness of cited answers
- Check usefulness of real-time agent evaluation logs
- Gather feedback on system performance
- Identify areas for improvement

Activity 5.5: Performance and Hallucination Metrics Evaluation

This activity measures system performance and validates the platform's core promise of reducing hallucinations.

- Measure total query response time, including any web fallback
- Evaluate citation accuracy and traceability rate
- Track frequency of fallback triggering versus pure local retrieval
- Monitor API response times (embedding service, web fallback)
- Ensure system stability under repeated runs

Folder Structure:

```
docutrust/
  ■■■ backend/
  ■   ■■■ main.py
  ■   ■■■ agents/
  ■   ■   ■■■ retriever_agent.py
  ■   ■   ■■■ grading_agent.py
  ■   ■   ■■■ query_rewriter_agent.py
  ■   ■■■ services/
  ■   ■   ■■■ embedding_service.py
  ■   ■   ■■■ citation_service.py
  ■   ■   ■■■ mongodb_service.py
  ■   ■■■ models/
  ■   ■■■ routes/
  ■   ■■■ requirements.txt
  ■■■ frontend/
  ■   ■■■ index.html
  ■   ■■■ styles/
  ■   ■■■ scripts/
  ■■■ outputs/
  ■■■ .env
```

Outputs:

Landing / Split-Pane Interface:

(Corporate split-pane UI showing PDF upload on the left and the validated answer panel on the right)

Document Upload and Chunking View:

(Multi-page PDF drag-and-drop with structural chunk extraction in progress)

Real-Time Agent Evaluation Log:

(Live step-by-step view of Retriever, Grading, and Query-Rewriter agent activity)

Validated, Cited Answer Output:

(Final answer displayed with strict, traceable citations to source documents)

Fallback Triggered View:

(Indicator showing that local retrieval was insufficient and a web fallback was used)